MUNIM: A CONTEXT-AWARE WHATSAPP SHOPPING ASSISTANT USING RETRIEVAL-AUGMENTED GENERATION

A

MINOR PROJECT REPORT

Submitted by

Saumya Aggarwal (08914812723)

Samaksh Azare (05214812723)

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND TECHNOLOGY

Under the Guidance of

[Guide name]

([Designation], Department of CST)

Department of Computer Science and Technology

Maharaja Agrasen Institute of Technology,

PSP area, Sector – 22, Rohini, New Delhi – 110085

(Affiliated to Guru Gobind Singh Indraprastha University, New Delhi)

DEC 2026

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Computer Science and Technology

CERTIFICATE

This is to certify that this MINOR project report "Munim: A Context-Aware WhatsApp Shopping Assistant Using Retrieval-Augmented Generation" is submitted by Saumya Aggarwal (08914812723) and Samaksh Azare (05214812723), who carried out the project work under my/our supervision.

I/We approve this MINOR project for submission.

Prof. (Dr.) Pooja Gupta
(Professor & HOD, CST)

[Guide name]
([Designation], Department of CST)
(Project Guide)

ABSTRACT

Small businesses in India increasingly sell through WhatsApp, but by hand: the catalogue, the cart and the payment live somewhere else, and every conversation needs a person. A conventional chatbot does not solve this, because shopping requests arrive as meaning rather than keywords (“something for my sister's mehendi, nothing too heavy, under 3000”), because language models invent prices and products, and because the WhatsApp Cloud API identifies a customer only by a phone number.

This project, Munim, is a WhatsApp shopping assistant that lets a customer discover, select and pay for products entirely inside a chat, with the same cart, account and orders on a companion web storefront. Requests are answered by Retrieval-Augmented Generation: the catalogue is embedded with EmbeddingGemma-300M (768 dimensions) and ranked by meaning with pgvector inside PostgreSQL, after which gender, garment, budget, colour and stock are enforced exactly in code. A tool-using language model (gpt-oss-20b via Groq) decides what to do next, but every product, price, cart and total shown to the customer is rendered and verified by code. Payments use Razorpay with signature verification and a row-locked, idempotent confirmation; a single-use code links a website account to a phone number; and the store owner improves the assistant by rating its answers, without retraining.

After the first defense the system was extended to serve several brands on one deployment, with per-store data isolation, encrypted credentials, verified webhook signatures, schema migrations and a Shopify catalogue importer. Every message is now traced, and an automatic grader plays 63 real-world conversations through the bot.

On 2 November 2026 all 13 automated test suites passed (552 checks); the retrieval evaluation passed 56 of 56 phrasings with 10 of 10 owner ratings held; and the conversation evaluation passed 60 of 63 cases (913 of 917 checks), at a median 2.6 seconds per reply and about ₹1.36 of model cost per 100 conversations. The work supports UN Sustainable Development Goals 8, 9 and 10 by giving small sellers affordable conversational commerce that customers can use in their own words.

Keywords: conversational commerce, WhatsApp Cloud API, retrieval-augmented generation, sentence embeddings, pgvector, LLM tool calling, grounding, multi-tenancy, Razorpay

ACKNOWLEDGEMENT

It gives us immense pleasure to express our deepest sense of gratitude and sincere thanks to our respected guide [Guide name], [Designation], Department of CST, MAIT Delhi, for their valuable guidance, encouragement and help in completing this work. Their useful suggestions for this whole work and co-operative behaviour are sincerely acknowledged.

We are also grateful to Prof. (Dr.) Pooja Gupta, Professor and Head of the Department of Computer Science and Technology, and to our teachers for their constant support and guidance.

We also wish to express our indebtedness to our parents and family members, whose blessings and support always helped us face the challenges ahead.

Place: Delhi — Saumya Aggarwal (08914812723)

Date: — Samaksh Azare (05214812723)

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Objectives 2

1.5 Scope 3

1.6 Organisation of the Report 3

CHAPTER 2 LITERATURE SURVEY 4

2.1 Conversational Commerce 4

2.2 Retrieval-Augmented Generation 4

2.3 Sentence Embeddings and Vector Search 4

2.4 Tool-Using Language Models and Grounding 5

2.5 Comparison with Existing Approaches 5

2.6 Research Gap 5

CHAPTER 3 RESEARCH AND APPROACH 6

3.1 Methodology 6

3.2 Requirements 7

3.3 System Architecture 8

3.4 Data Flow and Use Cases 9

3.5 Data Model 11

3.6 Retrieval 12

3.7 The Conversational Assistant and Grounding 13

3.8 Cart, Checkout and Payments 14

3.9 Account Linking and Live Sync 15

3.10 Owner Training 15

3.11 Multi-Store Operation (Phase P1) 15

3.12 Tracing and Conversation Evaluation (Phase P2) 15
CHAPTER 4 RESULTS 17
4.1 Prototype 17
4.2 Automated Tests 19
4.3 Retrieval Evaluation 20
4.4 Conversation Evaluation 20
4.5 Objectives Achieved 21
CHAPTER 5 CONTRIBUTION TO SOCIETY AND THE UN SDGS 22
CHAPTER 6 CONCLUSION, SUMMARY AND FUTURE SCOPE 23
6.1 Summary 23
6.2 Conclusion 23
6.3 Limitations 23
6.4 Future Scope 24
REFERENCES 25
APPENDICES 26
Appendix A: Running the System 26
Appendix B: Environment Variables 26
Appendix C: Team Contributions 26
Appendix D: Repository Layout 27
PROOF OF RESEARCH PAPER PUBLISHING 28

LIST OF TABLES

Table 1.1: Project objectives and their status 2
Table 1.2: Scope of the project 3
Table 2.1: Comparison of conversational shopping approaches 5
Table 3.1: Development timeline 6
Table 3.2: Hardware and software requirements 8
Table 3.3: Technology stack 9
Table 3.4: Database tables 11
Table 4.1: Sample conversation with the running system (2 Nov 2026) 18
Table 4.2: Automated test results (2 Nov 2026) 19
Table 4.3: Embedding model comparison 20
Table 4.4: Conversation evaluation by tag (a case can carry several tags) 20
Table 4.5: Cost and speed of the assistant 21
Table 5.1: Mapping of objectives to the UN Sustainable Development Goals 22
Table 6.1: Roadmap after the second defense 24
Table 6.2: Environment variables (secrets are never committed) 26
Table 6.3: Contribution of each team member 26

LIST OF FIGURES

Figure 3.1: PERT chart of the Phase 1 sprint 7
Figure 3.2: System architecture 8
Figure 3.3: Data flow diagram, level 0 (context diagram) 10
Figure 3.4: Data flow diagram, level 1 10
Figure 3.5: Use case diagram 11
Figure 4.1: Site search for “black kurta for office”: colour, garment and occasion understood 17
Figure 4.2: Product page with sizes, Razorpay payment and “Ask about this on WhatsApp” 18
Figure 4.3: Shop listing with filters by gender, category, price and function 18

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviation	Meaning
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CTA	Call to action (a WhatsApp button that opens a URL)
DFD	Data Flow Diagram
HMAC	Hash-based Message Authentication Code
HNSW	Hierarchical Navigable Small World (approximate index)
JSONB	Binary JSON column type in PostgreSQL
LLM	Large Language Model
MRP	Maximum Retail Price
MSME	Micro, Small and Medium Enterprises
ONNX	Open Neural Network Exchange (model format)
ORM	Object-Relational Mapper
p50 / p95	Median / 95th-percentile latency
PERT	Program Evaluation and Review Technique
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal (United Nations)
SKU	Stock Keeping Unit
UPI	Unified Payments Interface
<=>	pgvector's cosine-distance operator
d(q, p)	Cosine distance between query vector q and product vector p, $1 - \cos \theta$

CHAPTER 1
INTRODUCTION

1.1 Background

WhatsApp is the most used messaging application in India, and a large number of small businesses already use it to sell: a customer asks for something, the seller replies with photos, the two agree on a size and price, and payment happens through a UPI transfer or on delivery. The conversation works because the seller understands what the customer means. It does not scale, because every message needs that seller, and the catalogue, the order and the payment are kept by hand.

Large e-commerce companies solve the scale problem with websites, apps and search engines, but those put the customer in a form-driven interface rather than a conversation. This project asks whether a small store can have both: a conversation in the customer's own words, backed by a real catalogue, cart and payment system.

1.2 Problem Statement

Automating conversational shopping with a conventional chatbot runs into four specific problems:

1. **Natural-language requests are not database queries.** “Something for my sister's mehendi, nothing too heavy, under 3000” carries an occasion, a relationship, a style preference and a budget, and contains no keyword that matches a product title.
2. **Language models hallucinate.** A general-purpose model asked to recommend products may invent items, prices or discounts, which is unacceptable where money is involved. (During development the model once told a customer their total was ₹17,498 for a cart worth ₹10,798.)
3. **WhatsApp carries no identity beyond a phone number.** The Cloud API delivers the sender's number and the message text: no login, cookie or session. A customer signed in on the store's website is a stranger to the bot.
4. **A conversation is not a checkout.** Selection, sizes, cart, delivery address and payment confirmation must be handled reliably inside free-form chat, and must stay consistent with the website.

1.3 Motivation

- A real and visible need: WhatsApp commerce is how many Indian small sellers already work, informally.
- It applies current techniques (sentence embeddings, vector search, retrieval-augmented generation, LLM tool calling) to a problem where their weaknesses matter and must be engineered around.
- It is a full-stack engineering problem: webhooks, asynchronous APIs, relational design with transactions and constraints, payment-gateway integration, security and testing.
- After the first defense a real pilot became available: a friend's Shopify store selling T-shirts and workout clothes, which turned the project into a multi-brand product.

1.4 Objectives

The objectives set in the synopsis, and the two added after the first defense, are listed in Table 1.1. Each is revisited in Chapter 4.

Table 1.1: Project objectives and their status

No.	Objective	Status (2 Nov 2026)
O1	A WhatsApp chatbot on the official Cloud API that understands free-text shopping requests and replies with relevant products, photos and prices	Met
O2	A Retrieval-Augmented Generation pipeline: embed the catalogue, retrieve by meaning, enforce gender, category, budget and stock exactly	Met; colour, fabric and fit added in P2
O3	A conversational assistant that asks when a request is vague, remembers context and uses tools instead of inventing answers	Met
O4	Factual correctness: every product, price and total shown comes from the database	Met; verified in code
O5	A complete purchase in chat: select, size, cart, address, Razorpay payment, confirmation on WhatsApp	Met
O6	A web storefront sharing cart, orders and search with the bot, kept in sync live	Met
O7	Secure linking of a website account to a WhatsApp number, for personalised replies	Met
O8	Owner training: improve the assistant by rating answers and products, without retraining a model	Met
O9	Serve several brands on one deployment with isolated data (added after the first defense)	Met (phase P1)

O10	Measure conversation quality automatically on real-world cases (added after the first defense)	Met in part (P2, 4 of 6 steps)

1.5 Scope

Table 1.2: Scope of the project

In scope	**Out of scope (future work)**
Text conversations on WhatsApp; an 84-product demo store (Indian ethnic wear) and a second, invented activewear brand used in tests	Voice notes and photo search (phase P6)
Cart, address, Razorpay test-mode payments, order tracking	Live payments, stock reservation, refunds
Several stores on one deployment; a Shopify public-catalogue importer	A published Shopify app and hosted deployment (phase P3)
Owner training page; automated tests and conversation evaluation	A brand dashboard, human hand-off (P4)

1.6 Organisation of the Report

Chapter 2 surveys the literature and existing systems. Chapter 3 describes the approach: methodology, requirements, architecture, data flow, the data model and each subsystem, with code. Chapter 4 presents the results of testing and evaluation. Chapter 5 maps the objectives to the UN Sustainable Development Goals. Chapter 6 concludes and sets out the future scope. The appendices give setup instructions and the team's contributions.

CHAPTER 2
LITERATURE SURVEY

2.1 Conversational Commerce

Conversational commerce describes buying and selling through messaging interfaces rather than web forms. In India it has grown on WhatsApp because the application is already installed and trusted. Meta offers the WhatsApp Business app for small sellers, which provides catalogues and quick replies but leaves every conversation to a person, and the WhatsApp Cloud API [5] for programmatic access, which delivers each inbound message to a business's webhook and lets it reply with text, images and interactive buttons. In 2026 Meta also introduced an AI assistant inside the Business app; it answers from a business's profile but does not hold a real cart or place orders in the business's own checkout. Third-party platforms (marketing and broadcast tools on the Cloud API) focus mostly on templated messages and flows built by hand.

2.2 Retrieval-Augmented Generation

Lewis et al. [1] proposed retrieval-augmented generation (RAG): instead of relying on what a language model memorised in training, relevant documents are retrieved from an external store and placed in the model's context, so that its output is grounded in them. RAG suits product recommendation well, because a catalogue changes daily and a model cannot be retrained for each change. Its known weakness is that retrieval by meaning ranks documents but does not enforce hard constraints such as a budget or a gender; this project found that weakness directly (Section 3.6).

2.3 Sentence Embeddings and Vector Search

Sentence-BERT [3] showed that siamese transformer networks produce sentence vectors whose cosine similarity reflects meaning, making semantic search fast. The first version of this project used all-MiniLM-L6-v2, a small 384-dimensional English model of this family. EmbeddingGemma [2] is a 300-million-parameter multilingual embedding model from Google that produces 768-dimensional vectors and accepts task prompts (document, query, similarity); it runs on a CPU, which keeps the system free to run. For storage and search, pgvector [7] adds a vector column type and distance operators to PostgreSQL, so

that vectors live next to the relational data and a single SQL query can both filter and rank. Approximate indexes such as HNSW trade completeness for speed; at a catalogue size of a few hundred products per store an exact scan is fast and never misses an item.

2.4 Tool-Using Language Models and Grounding

Schick et al. [4] showed that language models can learn to call external tools (APIs) and use their results. Current models expose this as function calling: the application declares tools with typed arguments and the model returns which tool to call. This lets the model decide what to do while the application decides what is true. The literature on hallucination shows that instructions alone do not stop a model stating unsupported facts; the reliable approach is to verify outputs against trusted data, which is the design used here (Section 3.7).

2.5 Comparison with Existing Approaches

Table 2.1: Comparison of conversational shopping approaches

Approach	Understands free text	Real cart and checkout	Facts grounded	Shares state with website
Seller replying by hand on WhatsApp Business app	Yes (a person)	No	Yes (a person)	No
Rule-based / flow-builder chatbot	No (menus, keywords)	Sometimes	Yes	Rarely
General LLM chatbot on a WhatsApp number	Yes	No	No (hallucinates)	No
Meta's AI in the Business app (2026)	Yes	No	From the profile	No
Munim (this project)	**Yes (RAG + LLM)**	**Yes (cart, Razorpay)**	**Yes (verified in code)**	**Yes (one database)**

2.6 Research Gap

Existing tools either understand language but cannot be trusted with prices and orders, or are reliable but cannot understand a request written in the customer's own words. The gap this project addresses is a system that does both: it uses a language model and semantic retrieval for understanding, but keeps every fact, identity binding and payment confirmation as explicit, verified system state.

CHAPTER 3
RESEARCH AND APPROACH

3.1 Methodology

The project followed an iterative and incremental model. Each increment delivered a working end-to-end slice (for example, “message in, three products out”) that was tried on a real phone and covered by an automated test suite before the next began. Phase 1 was a six-day sprint (18–23 September 2026) by two developers, split into a “buying” half and a “finding” half joined by written interface contracts. After the first defense on 23 September the work was planned as a product roadmap of phases P0 to P7, with a decision log recording each choice and the options rejected.

From phase P2 the method became evaluation-driven: 54 real logged conversations were analysed, turned into 63 graded test cases, and the bot was changed only when a case showed what to fix. Expectations are written for the behaviour wanted, so a failing case is a to-do rather than a broken test.

Table 3.1: Development timeline

Period	Increment	Main outcome
18–23 Sep	Phase 1 sprint	Webhook, retrieval, assistant, cart, Razorpay, storefront, account linking, live sync, owner training
23 Sep	First defense	Graded prototype
28 Sep	P0: cleanup	One command runs every suite; logged conversations exported for evaluation
1–2 Oct	P1: multi-store	Stores and tenancy, catalogue and vectors in Postgres (pgvector), EmbeddingGemma, Alembic, signed webhooks, Shopify importer
3–15 Oct	P2: bot quality (steps 1–4)	Tracing, conversation evaluation harness, checkout in plain words, any brand's catalogue, colour/fabric/fit filters

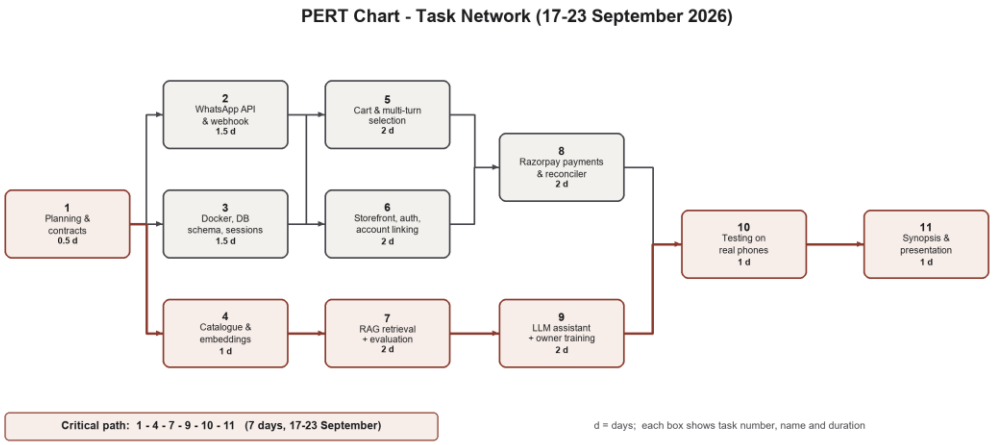


Figure 3.1: PERT chart of the Phase 1 sprint

3.2 Requirements

3.2.1 Functional requirements

- Receive WhatsApp messages through Meta's webhook, verify their signature and reply within seconds.
- Understand free-text requests and return matching products with photos, prices and links.
- Resolve follow-ups: numbers (“2”), names (“the blush pink one”), sizes, and questions about a shown item.
- Maintain a cart shared by WhatsApp and the website; take a delivery address; accept payment through Razorpay.
- Link a website account to a WhatsApp number; personalise replies with name, purchase history and sizes.
- Let the store owner rate answers and products; let customers hide items for themselves.
- Serve several stores, each with its own catalogue, customers, credentials and wording.

3.2.2 Non-functional requirements

- **Correctness:** no invented product, price, total, size or cart change may reach a customer.
- **Reliability:** the webhook always returns HTTP 200 to Meta (except a bad signature, 403); duplicate deliveries are ignored; a payment produces exactly one confirmation; the shop keeps answering if the language model is down.

- **Security:** secrets only in environment variables or encrypted columns; webhook and payment signatures verified; tools bound to the sender's account.
- **Cost:** runs on a laptop for the demo with free tiers; hosting target under ₹1,000 a month.
- **Testability:** every subsystem has an automated suite that needs no paid API.

3.2.3 Hardware and software

Table 3.2: Hardware and software requirements

Item	Specification
Development machine	Windows 11 laptop, CPU only (no GPU needed); 8 GB RAM or more
Runtime	Python 3.12, FastAPI + Uvicorn, Docker Desktop (PostgreSQL 15 with pgvector), Node.js (one test)
External services	Meta WhatsApp Cloud API (test number), Groq API (LLM), Razorpay (test mode), ngrok (public HTTPS tunnel)
Client	Any phone with WhatsApp; any modern browser

3.3 System Architecture

The system has two front doors, a WhatsApp number and a web storefront, served by one FastAPI application (Figure 3.2). Both read and write the same PostgreSQL database, which since phase P1 also holds the product vectors. The embedding model runs inside the application process. Meta, Razorpay and Groq are the external services.

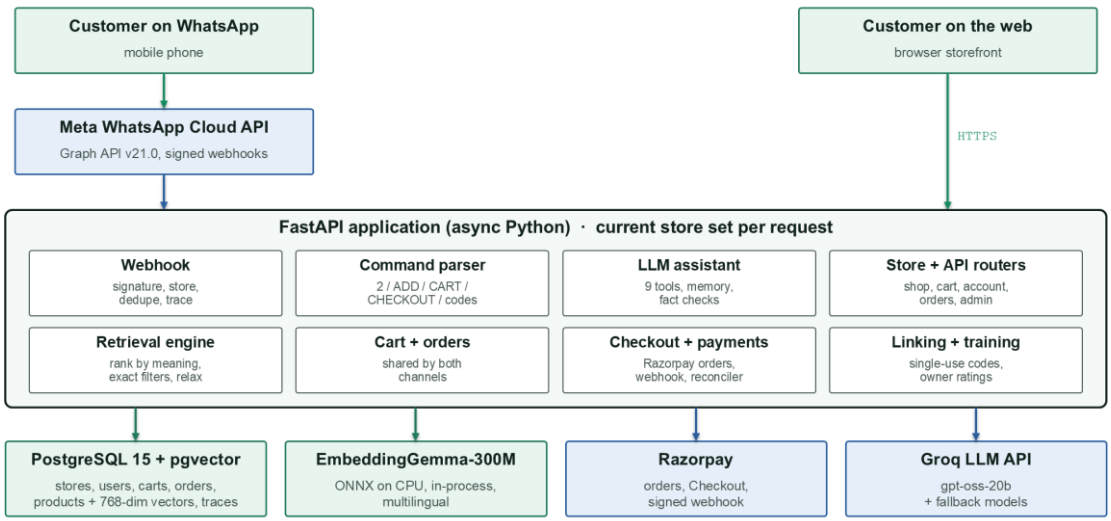


Figure 3.2: System architecture

Table 3.3: Technology stack

Layer	Technology	Reason for the choice
Channels	WhatsApp Cloud API v21.0; Jinja2 templates with compiled Tailwind CSS	Official API (no ban risk); the site works offline at a venue
Application	Python, FastAPI, Uvicorn, SQLModel/Pydantic, httpx	Async suits a webhook that must reply at once while calling slow services
Retrieval	EmbeddingGemma-300M (int8 ONNX on CPU), pgvector	Multilingual, free, cannot be rate-limited; one database for data and vectors
Language model	Groq gpt-oss-20b with tools; fallbacks gpt-oss-120b, qwen3.8-27b	OpenAI-compatible, fast, free tier; one module (llm.py) with a role per job
Data	PostgreSQL 15, Alembic migrations	Constraints, transactions, row locks, NUMERIC money; versioned schema
Payments	Razorpay Standard Checkout and webhooks (test mode)	UPI, cards and netbanking in India; a thin httpx wrapper, no synchronous SDK
Security	bcrypt, signed cookies, HMAC-SHA256, Fernet	Password hashing, webhook/payment signatures, encrypted store secrets
Operations	Docker Compose, ngrok	Same database image everywhere; public HTTPS URL for Meta

3.4 Data Flow and Use Cases

The context diagram (Figure 3.3) shows the system with the people and services it exchanges data with. Level 1 (Figure 3.4) opens it into seven processes and their data stores. Since phase P1 the product vectors (D4) and the catalogue (D5) are tables in the same PostgreSQL database.

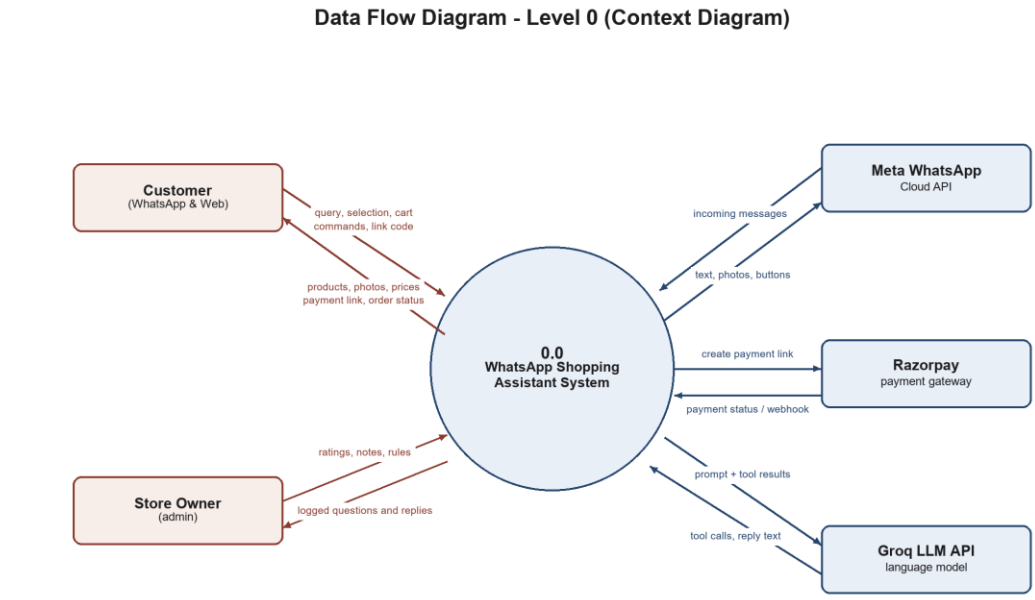


Figure 3.3: Data flow diagram, level 0 (context diagram)

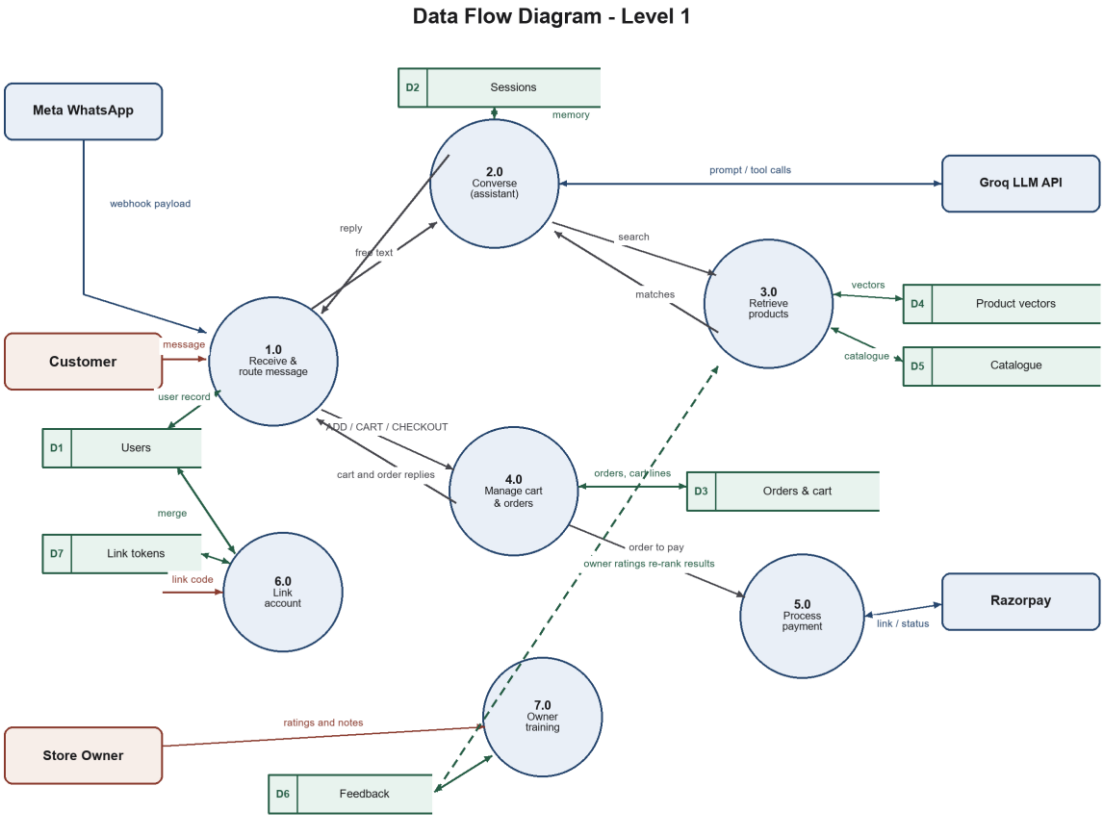


Figure 3.4: Data flow diagram, level 1

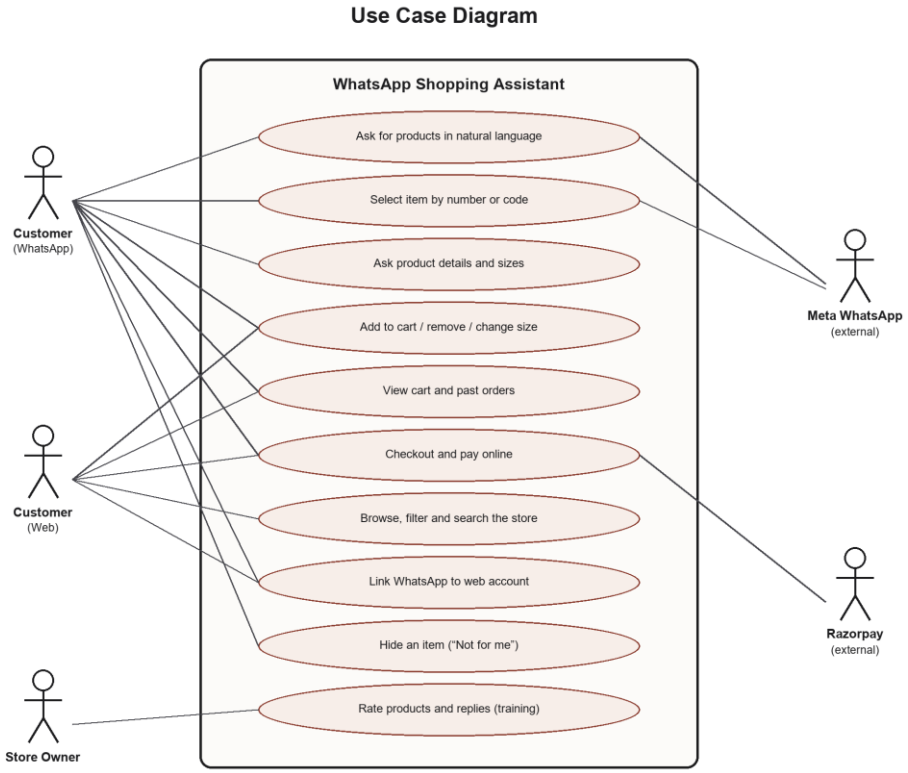


Figure 3.5: Use case diagram

3.5 Data Model

The schema has thirteen tables, managed by Alembic (six revisions, applied on every start). Every table that holds a customer, an order, a reply or a product carries a `store_id`.

Table 3.4: Database tables

Table	Holds	Constraint worth noting
stores	Each brand: slug, code prefix, encrypted WhatsApp credentials, brand texts (JSONB)	Secrets encrypted with Fernet
products	One row per style per colour, with its 768-dim vector and model name	Unique (store, code); vector compared only with the same model
product_variants	One row per size: stock, price, Shopify variant id	Belongs to a product
users	One identity per person per store, either channel	Email or WhatsApp number required; unique per store
sessions	Conversation state: last products shown, selection, messages, pending address	One active session per user
cart_items	The shared cart	Unique (user, product, size); quantity > 0
orders / order_items	Order header (status, total, Razorpay ids,	Money as

	address) and lines with copied name and price	NUMERIC(10,2)
link_tokens	Single-use account-linking codes	Used once, expire after 10 minutes
bot_replies / feedback	Every free-text answer, and owner ratings with the question's embedding	Owner training log
processed_messages	Ids of WhatsApp messages already handled	Stops Meta's redeliveries
turn_traces	One row per inbound message: path, LLM calls, tools, searches, guards, reply	Used by the evaluation harness

3.6 Retrieval

Ingestion. For each product one text is built from its title, gender and category, fabric, colour and pattern, fit, occasions, description and highlights. Price and stock are deliberately not embedded: they are filters, not meaning. The text is embedded with EmbeddingGemma's document prompt and stored in the product row.

Query. The request is parsed for gender, garment words, budget, occasion, colour, fabric and fit, using a vocabulary built per store from its own catalogue (for example T-Shirts → tee, tshirt, t shirt). The request is embedded with the query prompt and one SQL statement ranks the store's whole catalogue by cosine distance (Listing 3.1). The constraints are then applied exactly in Python, because embeddings rank but do not constrain: the first version ranked a women's kurta set first for “groom outfit”. If nothing passes, constraints are relaxed in a fixed order (colour, budget, garment) and the reply says so. A distance cut-off (cosine distance 0.74) makes nonsense return nothing. Products tagged for the requested occasion come first, and owner ratings for similar questions re-rank the result.

Listing 3.1: Ranking a store's catalogue with pgvector (backend/catalog.py, trimmed)

```
def ranked_by_vector(vector, exclude=None, gender=None, limit=None):
    """(code, cosine distance) for the current store's products, nearest
first.
    An exact scan: never skips a product."""
    sql = ["SELECT code, embedding <=> CAST(:vector AS vector) AS distance
FROM products",
           "WHERE store_id = :store AND active AND embedding_model =
:model"]
    params = {"vector": str(list(vector)), "store": tenancy.store_id(),
              "model": embeddings.model_name()}
    if gender:
        sql.append("AND gender = :gender")
        params["gender"] = gender
    sql.append("ORDER BY distance, code")
    with session_scope() as db:
        return [(code, float(d)) for code, d in db.exec(text("
".join(sql)).bindparams(**params)).all()]
```

The cosine distance between a query vector q and a product vector p is d(q, p) = 1 − (q · p) / (‖q‖ ‖p‖), which pgvector's <=> operator computes.

3.7 The Conversational Assistant and Grounding

Messages that are exact commands (a number, ADD 42, CART, CHECKOUT, a product code) are parsed by code and never reach the language model. Free text goes to the model with the last eight messages, the customer's real cart, profile and order history, and nine tools: search_products, get_product_details, get_my_orders, get_my_cart, add_to_cart, remove_from_cart, hide_product, checkout and send_reply. The tools are bound to the sender's account; the model cannot pass a user id.

Trust is enforced in code, never by instruction alone:

- Product lists, captions, prices, photos, carts and orders are rendered by code; send_reply only names product ids, which must exist.
- Any rupee amount in the model's own sentence must appear in the catalogue, the customer's words or a tool result. Otherwise the model is told to try again once, and then the sentence is removed (Listing 3.2).
- A reply that claims “added to your cart” without a successful cart tool, describes a payment button, or names a size the product does not come in is sent back once and then cut. Cart confirmations are written by code from a ledger of what the cart tools actually did.
- add_to_cart needs the customer's own message to ask for it, and a size they gave.
- A reply that looks like program code is replaced by the store's off-topic line: it is a shop assistant, not a chatbot.
- If the model is rate-limited the next model is tried, then a short wait, then a plain retrieval reply with no model.

Listing 3.2: Verifying rupee amounts in the model's reply (backend/bot/assistant.py, trimmed)

```
def _unverified_amounts(message: str, evidence: list[str]) -> list[str]:
    """Rupee amounts in the message that appear nowhere we can vouch
for."""
    known = {round(p["price"]) for p in catalog.get_all()} |
{round(p["mrp"]) for p in catalog.get_all()}
    for text in evidence:      # the customer's words, the real cart, tool
results
        known |= {_value(n, u) for n, u in _NUMBER_IN_TEXT.findall(text or
"")}
    wrong = []
```

```
    for number, unit in _AMOUNT.findall(message):
        if _value(number, unit) not in known:
            wrong.append(f"Rs {number}{unit or ''}")
    return list(dict.fromkeys(wrong))

unverified = _unverified_amounts(reply.message, evidence)
if unverified and not fact_checked:
    fact_checked = True                          # one retry
    return None, f"ERROR: {', '.join(unverified)} does not match our
records. ..."
if unverified:                                   # then the sentence is
dropped
    reply.message = _drop_sentences_with(reply.message, unverified)
```

3.8 Cart, Checkout and Payments

The cart is one table shared by both channels. CHECKOUT (or “i wanna pay”, recognised in code, including misspellings within two edits) checks sizes, asks for a delivery address if none is saved (parsing a courier-style message and asking for missing fields one at a time), creates an order and a Razorpay order, and sends a Pay Now button that opens the store's payment page. Three independent paths confirm a payment: the signature Razorpay Checkout returns to the browser (HMAC-SHA256 of “order_id|payment_id” with the key secret), Razorpay's signed webhook, and a background reconciler that polls Razorpay every five seconds for unpaid orders from the last hour. Any one is enough; a row lock ensures only the first sends a confirmation (Listing 3.3).

Listing 3.3: Idempotent, row-locked payment capture (backend/repository.py, trimmed)

```
def mark_order_paid(order_id, razorpay_payment_id, razorpay_order_id=None)
-> bool:
    """Record a captured payment. True only the first time for an
order."""
    with session_scope() as db:
        order = db.exec(select(Order).where(Order.order_id ==
order_id).with_for_update()).first()
        if order is None or order.status == "captured":
            return False
        order.status = "captured"
        order.razorpay_payment_id = razorpay_payment_id
        order.captured_at = datetime.now(timezone.utc)
        # only the ordered quantities leave the cart, in the same
transaction
        ...
    return True
```

3.9 Account Linking and Live Sync

A browser session cannot reach the bot, because Meta delivers only a phone number and a message. The account page therefore mints a single-use code valid for ten minutes and a wa.me link that pre-fills it in WhatsApp. When the code arrives from a number, the webhook binds that number to the account in one transaction; if the number already had a bot-created account, its orders and sessions are moved to the website account and the duplicate is deleted. Signed-in pages poll a small fingerprint of the cart and recent orders every two seconds and re-render in place when it changes, so an item added in WhatsApp appears on the open website within two seconds.

3.10 Owner Training

Every free-text answer is logged. On the admin page the owner rates products (✓/✗) for a question and rates whole replies. A product rating re-ranks that product for questions whose embedding is similar (cosine ≥ 0.87 with EmbeddingGemma), in both the bot and the site search; an approved reply becomes an example for similar questions; a note on a rejected reply becomes a rule in the prompt. The model itself never changes, only what it is shown.

3.11 Multi-Store Operation (Phase P1)

The current store is held in a Python ContextVar; unset means the demo store, so existing code paths continue to work. The webhook sets it from the phone-number id in Meta's payload, and checks each change against the X-Hub-Signature-256 header (HMAC-SHA256 of the raw body) with that store's app secret; a request where nothing verifies receives HTTP 403. Each store's WhatsApp credentials are encrypted with Fernet. Brand texts (name, description, what it sells, policies, off-topic line) live on the store and render the assistant's prompt, greeting and help text. A connector imports a Shopify shop's public catalogue at one request per second, mapping each product colour to a product and each size to a variant.

3.12 Tracing and Conversation Evaluation (Phase P2)

Every inbound message records one trace: the exit that answered it, every LLM call (model, time, tokens, cost), every tool call and search, every guard that intervened, and

the reply as sent. Recording is proven never to change a reply: a test plays 60 turns with tracing made to fail and obtains identical replies and database state.

The evaluation harness plays YAML-defined conversations through the real message routing, each with a throwaway customer, and grades every turn by code from its trace and the before/after database state: eleven checks on every turn (for example: amounts real, claims backed by tool calls, no code, no fallback, sizes real) plus each case's own expectations. A run tests one model with no fallbacks, and paces itself to the free tier's limits.

CHAPTER 4

RESULTS

4.1 Prototype

The working prototype was demonstrated end to end on a real phone, including a test-mode payment. Figures 4.1 to 4.3 show the storefront, which shares its search engine with the bot, and Table 4.1 shows replies produced by the real system on 2 November 2026 through the local webhook simulator (the same code path as WhatsApp; photos are sent as separate image messages).

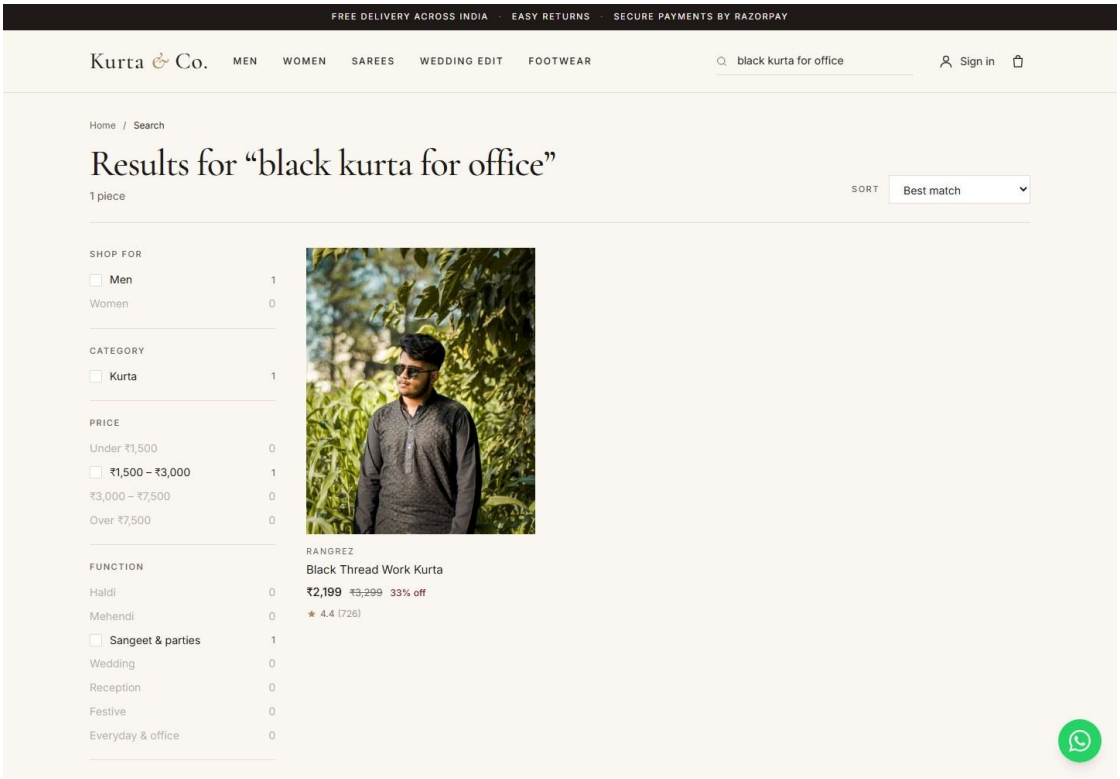


Figure 4.1: Site search for “black kurta for office”: colour, garment and occasion understood

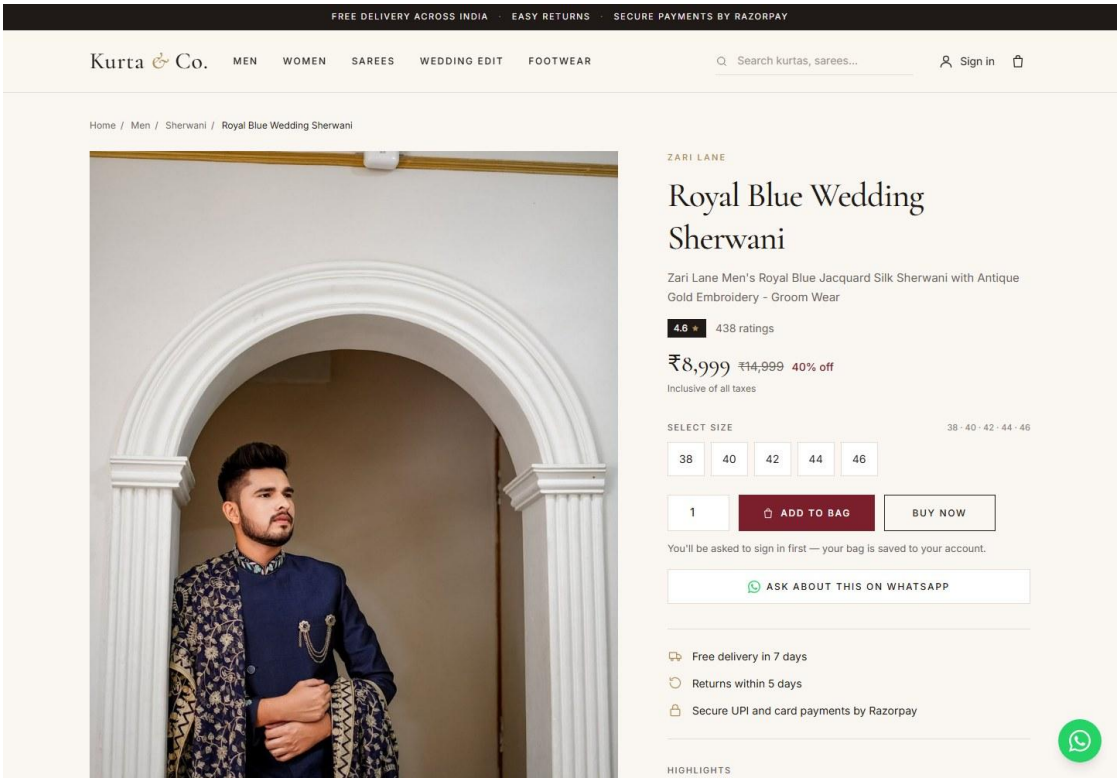


Figure 4.2: Product page with sizes, Razorpay payment and "Ask about this on WhatsApp"

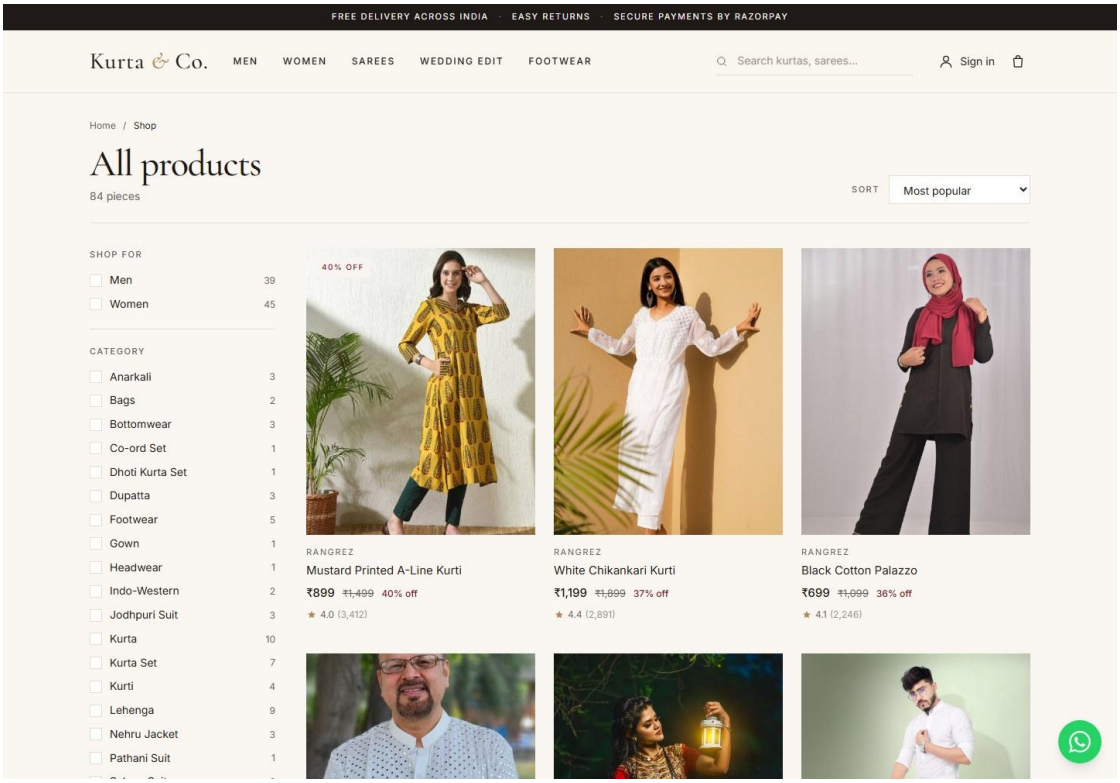


Figure 4.3: Shop listing with filters by gender, category, price and function

Table 4.1: Sample conversation with the running system (2 Nov 2026)

Customer message	Reply from Munim (abridged)
sherwani for my wedding under	"Here are three great options for your wedding sherwani under

10000	₹10,000.” 1. Royal Blue Wedding Sherwani, Rs. 8,999; 2. Midnight Teal Sherwani, Rs. 7,999; 3. Maroon Sequin Stripe Sherwani, Rs. 9,499 (with photos and links). “Reply 1–3 to choose · CART to see your cart”
black kurta for office	“Who will be wearing the black kurta?” (it asks, because the wearer is unknown)
for me, i'm a man, under 2500	1. Black Thread Work Kurta, Rs. 2,199: “A black cotton-silk kurta with tonal black thread work across the yoke …”
ADD 42	“Black Thread Work Kurta comes in S, M, L, XL. Reply with your size, for example L.” (a size the item does not come in is refused by code)
can u write a python code for 2+2	“I'm here to help with your ethnic wear, not coding.”

4.2 Automated Tests

All suites are run with one command and need no paid service: the language model is replaced by a scripted fake, and Razorpay and WhatsApp are faked. They do need PostgreSQL, because they exercise the real constraints. On 2 November 2026 all 13 suites passed in 166 seconds.

Table 4.2: Automated test results (2 Nov 2026)

Suite	Checks	What it proves
test_cart.py	126	Cart operations, concurrent adds, checkout, the same cart across channels
test_assistant.py	96	Tool binding, memory, code-rendered lists, consent before adding, fact checks, fallbacks
test_tenancy.py	62	Isolation of customers, carts, orders, catalogue, vectors, search, codes, credentials; signatures
test_payments.py	61	Signatures, one order per checkout, forged callbacks rejected, simultaneous confirmations
test_eval_harness.py	44	The conversation grader itself
test_brands.py	40	A second (activewear) brand through the Shopify importer; the demo unchanged
test_store.py	36	Filters, listings, search, sign-in redirects, address validation
test_training.py	28	Owner ratings re-rank similar questions; undo; personal hides
test_live.py	20	Live-sync endpoint and fingerprint
test_linking.py	18	Chat first, then sign-up, then the merge; replayed codes refused
test_live_js.mjs	15	The browser live-sync script in Node against a fake DOM
test_trace_neutrality.py	6	60 turns with tracing failing: identical replies and state
Total	**552**	**All passed**

4.3 Retrieval Evaluation

`scripts/eval_retrieval.py` holds 56 phrasings, including 10 colour, fabric and context cases, each with the constraint every result must satisfy; a case passes only if every returned product fits, because "one of three was right" hides the miss a customer actually sees. It also replays every owner rating as a regression. Result: **56 of 56 cases and 10 of 10 owner ratings**, with no language model involved. The keyword filters of the earliest version scored 8 of 10 on an easier ten-case set and returned nothing at all for two of them.

The embedding model was chosen by measurement (`scripts/eval_embeddings.py`, 2 October 2026):

Table 4.3: Embedding model comparison

Measure	all-MiniLM-L6-v2	EmbeddingGemma-300M
Dimensions, language	384, English	768, multilingual
English retrieval cases passed	46 / 46	46 / 46
Hinglish requests whose nearest product is on target	22%	60%
Nonsense separated from real requests	52 of 53 (overlap)	Fully (real ≤ 0.708, nonsense ≥ 0.777)
Distance cut-off chosen	0.775	0.74

4.4 Conversation Evaluation

The conversation harness was first run on 14 October 2026 with gpt-oss-20b on Groq's free tier: 47 of 63 cases passed. The 16 failures became the fix list for P2 step 2 (checkout in plain words never paid, an offered size ignored, malformed tool calls, colour not a filter, and others). After step 2, **60 of 63 cases and 913 of 917 checks passed**, with no regressions.

Table 4.4: Conversation evaluation by tag (a case can carry several tags)

Tag (cases)	Passed	Tag (cases)	Passed
cart (17)	17	size (11)	11
discovery (21)	18	checkout (9)	9
category (8)	7	lookup (8)	8
budget (6)	5	wearer (6)	5
typo (5)	5	address (4)	4
smalltalk (4)	4	colour (3)	2

consent (3)	3	off-topic (2)	2
media (2)	2	reorder (2)	2

Table 4.5: Cost and speed of the assistant

Metric (per model turn, gpt-oss-20b)	**Value**
Cases passed / checks passed	60 / 63; 913 / 917
Median and 95th-percentile reply time	2.6 s; 4.9 s
LLM calls per model turn	1.62
Input / output tokens per model turn	≈ 2,835 / 102
Model cost	≈ ₹1.36 per 100 conversations
WhatsApp messages per answer	1.62

The three cases still failing are recorded as work: “only male options” asks who the outfit is for again; accessories within a budget show the wrong items; and one colour case. Colour, fabric and fit became filters in step 4, after this baseline, and have not yet been re-graded with the model.

4.5 Objectives Achieved

Objectives O1 to O9 are met (Table 1.1). O10 is met in part: tracing and the evaluation harness exist and are in use; choosing the model by an A/B test and reducing messages per answer (P2 steps 5–6) remain.

CHAPTER 5
CONTRIBUTION TO SOCIETY AND THE UN SDGS

The United Nations' 2030 Agenda [9] sets 17 Sustainable Development Goals. Munim is a tool for small sellers and their customers, so its contribution is indirect; it will be measured in the pilot with a real store (P3). Table 5.1 maps the project's objectives to the goals and targets they support.

Table 5.1: Mapping of objectives to the UN Sustainable Development Goals

SDG and target	How Munim contributes	Objectives
SDG 8, Decent Work and Economic Growth. Targets 8.2 (productivity through technology) and 8.3 (support for micro and small enterprises)	Gives a small seller an always-available sales assistant that answers questions, shows photos and collects payment, a capability usually affordable only to large e-commerce firms; one seller can serve many more customers.	O1, O3, O5, O9
SDG 9, Industry, Innovation and Infrastructure. Targets 9.3 (integrate small enterprises into value chains and markets) and 9.c (access to ICT)	Connects a small brand's catalogue, cart and payments to the channel its customers already use; one deployment serves many brands for a few hundred rupees a month, and a Shopify importer brings an existing store in with one command.	O2, O5, O6, O9
SDG 10, Reduced Inequalities. Target 10.2 (social and economic inclusion of all)	Shopping by typing a sentence, in English or Hinglish, needs no app, password or website skills; the multilingual embedding model was chosen because it puts 60% of Hinglish requests on target against 22%.	O1, O3, O7, O10

Two further contributions are relevant. Payments inside the chat through a verified gateway (UPI and cards) replace sharing bank details over messages. Clear product information, enforced size choices and confirmed orders reduce wrong purchases and returned parcels, which supports responsible consumption (SDG 12). Finally, the project shows a pattern for using language models responsibly where money is involved: the model chooses actions, but code verifies every fact before it reaches a person.

CHAPTER 6

CONCLUSION, SUMMARY AND FUTURE SCOPE

6.1 Summary

Munim lets a customer shop a store entirely inside WhatsApp: describe a need in their own words, see real products with photos and prices, choose by number or name, add sizes to a cart, give a delivery address and pay, with the same cart, account and orders on the store's website. Requests are answered by retrieval-augmented generation over a PostgreSQL catalogue with pgvector and EmbeddingGemma, with hard constraints enforced in code. A tool-using language model decides what to do, while code renders and verifies every product, price, cart change and payment. Since the first defense the system serves several brands on one deployment, and its quality is measured automatically on 63 real-world conversations.

6.2 Conclusion

- **Semantic retrieval must be combined with exact constraints.** Embeddings rank by meaning but do not respect a budget or a gender; ranking first and filtering after, with honest relaxation, passed all 56 evaluation phrasings.
- **A language model can be made safe around money** by verifying its output against trusted data rather than instructing it: 913 of 917 graded checks pass, including that every amount is real and every cart claim is backed.
- **Identity can be bridged** between a browser and a phone number that carries no session with a single-use code, including the merge of an account the bot created earlier.
- **Measurement changes development:** the conversation harness turned “the bot feels better” into 47 → 60 of 63.

6.3 Limitations

- It runs on a laptop behind ngrok, on a Meta test number whose token expires every 24 hours and which can message only allow-listed phones.
- Stock is displayed but not reserved; Razorpay is in test mode; payments and the website serve the demo store only.

- One model on a free tier (8,000 tokens per minute per model) until the model A/B test.
- Evaluation uses our own cases from real logs, not click-through or purchase data from live customers.

6.4 Future Scope

Table 6.1: Roadmap after the second defense

Phase	Work
P2 (steps 5–6)	Choose the main model by an A/B test on our own conversations; send fewer messages per answer
P3	A Shopify app connector (cart into the brand's own checkout), deployment on a VPS (about ₹560/month), pilot go-live
P4	Brand dashboard with revenue driven; hand-off to a person
P5	Order status, exchanges, abandoned-checkout recovery
P6	Voice notes (speech-to-text for Indian languages) and search by photo
P7	Bot quality, part 2, on real pilot traffic

REFERENCES

[1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[2] H. S. Vera et al., “EmbeddingGemma: Powerful and lightweight text representations,” arXiv preprint arXiv:2509.20354, 2025.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[4] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] Meta Platforms, Inc., “WhatsApp Cloud API,” Meta for Developers. [Online]. Available: https://developers.facebook.com/docs/whatsapp/cloud-api

[6] Razorpay, “Standard Checkout” and “Webhooks,” Razorpay Docs. [Online]. Available: https://razorpay.com/docs/

[7] pgvector contributors, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[8] S. Ramírez, “FastAPI.” [Online]. Available: https://fastapi.tiangolo.com/

[9] United Nations General Assembly, “Transforming our world: the 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015.

[10] OpenAI, “gpt-oss-120b and gpt-oss-20b model card,” 2025. [Online]. Available: https://openai.com/index/introducing-gpt-oss/

[11] SQLAlchemy authors, “Alembic: A database migrations tool for SQLAlchemy.” [Online]. Available: https://alembic.sqlalchemy.org/

APPENDICES

Appendix A: Running the System

```
# 1. Database (PostgreSQL 15 with pgvector)
docker compose up -d

# 2. Application, from backend/  (migrations run on start)
.venv/Scripts/uvicorn main:app --port 8000

# 3. Public HTTPS URL for Meta's webhook (callback = <url>/webhook)
ngrok http --url=<your-domain>.ngrok-free.dev 8000

# Every test suite plus the retrieval evaluation, from the repository root
backend/.venv/Scripts/python scripts/run_all_tests.py

# Talk to the bot without a phone (server running)
backend/.venv/Scripts/python scripts/simulate_webhook.py --message "saree
for office"

# Add a brand on Shopify and import its catalogue
backend/.venv/Scripts/python scripts/stores.py create <slug> "<Name>" --
prefix XX --shop <domain>
backend/.venv/Scripts/python scripts/sync_catalog.py <slug>
```

Appendix B: Environment Variables

Table 6.2: Environment variables (secrets are never committed)

Variable	Purpose
WHATSAPP_ACCESS_TOKEN, WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_VERIFY_TOKEN, WHATSAPP_APP_SECRET	The demo store's WhatsApp Cloud API credentials and webhook signature secret
LLM_API_KEY, LLM_* (per role)	Language model endpoint, model and key for each role (chat, vision, speech, judge)
EMBEDDING_PROVIDER	gemma (default), minilm or openai
MUNIM_SECRET_KEYS	Fernet keys that encrypt stores' secrets (first encrypts, all decrypt)
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET	Razorpay test-mode keys
DATABASE_URL	PostgreSQL connection string

Appendix C: Team Contributions

Table 6.3: Contribution of each team member

Member	Roll No.	Contribution

Saumya Aggarwal	08914812723	Phase 1 “buying” half: FastAPI backend, WhatsApp webhook and messaging, PostgreSQL schema, cart and orders, Razorpay payments, account linking, storefront redesign and live sync, the conversational assistant and owner training. After the first defense: multi-store tenancy (P1), pgvector, EmbeddingGemma, Alembic, signed webhooks, the Shopify importer, tracing and the conversation evaluation harness (P2).
Samaksh Azare	05214812723	Phase 1 “finding” half: product embeddings and the first retrieval pipeline, filtered product retrieval, catalogue search, LLM-assisted and personalised recommendations, the search and recommendation pages, and the first version of the retrieval evaluation script.

Appendix D: Repository Layout

```
backend/            FastAPI app: routers, models, migrations, connectors
(Shopify, feed)
backend/bot/        assistant.py (LLM + tools), retrieval.py (search),
chat.py (plain path)
common/             shared Pydantic schemas, embeddings.py
scripts/            test suites, evaluations, sync_catalog.py, stores.py
evals/              conversation cases, configs, baselines
data/products.json  the demo catalogue (84 products)
```

PROOF OF RESEARCH PAPER PUBLISHING

[To be added: acceptance letter or publication proof of the research paper based on this project.]